Notes:

- HW1 graded, solutions posted
- HW2 due Oct 16
- Project description to be posted later today, proposals due Oct 21
- HW3 goes out Thursday

Last class: Reproducible research practices. Code should be:

- Readable
- Runnable
- Correct
  - code reviews, PRs
- Documented
- Open

Consider 3-step development process:

1. Write correct code.
2. Identify computational bottlenecks
3. Find faster implementations where needed

Later in the course, we'll talk about steps 2 and 3. For the second step, we'll look at using performance profiling to find the functions and operations that account for significant computational time. Focus on improving these.

How?

1. Use packages - look for performant implementations of common functions.
2. For Python code, tools like Numba (translates *some* Python code to fast machine code) or pybind (that allows us to "bind" together python and C++) offer good tools. But also need to consider whether there is simply a better way to express what we need to compute in our codes.

Today's focus: testing

Consider equipment used for physical lab experiments. All equipment is checked and calibrated (performance measured to compare **expected** responses to known inputs against **actual** responses) before being used in an experiment.

The same should be true for software used in computational experiments.

All nontrivial software should be suspected of having bugs. We test our software to give us *some* confidence that it is doing what we expect.

Our goal in writing software is not just to write the software, but also to write a comprehensive set of tests (known as the "test suite") covering, as best as possible, the functionality of the code.

Closely related to this philosophy is the idea of "defensive programming", where we include checks that user-provided input matches our expectations.

A standard way to do this in Python is using assertions and exceptions

In [1]:
```
def inverse(x):
    return 1/x
```

In [2]:
```
inverse(5)
```

Out[2]:
```
0.2
```

In [3]:
```
inverse(2.5)
```

Out[3]:
```
0.4
```

In [4]:
```
inverse('frog')
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[4], line 1
----> 1 inverse('frog')

Cell In[1], line 2, in inverse(x)
      1 def inverse(x):
----> 2     return 1/x

TypeError: unsupported operand type(s) for /: 'int' and 'str'
```

1/x only makes sense if x is a number!

Two ways of checking this:

In [5]:
```
def inverse_assert(x):
    assert type(x) == float or type(x) ==int
    return 1/x
```

In [6]:
```
inverse_assert(5)
```

Out[6]:
```
0.2
```

In [7]:
```
inverse_assert(2.5)
```

Out[7]:
```
0.4
```

In [8]:
```
inverse_assert('frog')
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[8], line 1
----> 1 inverse_assert('frog')

Cell In[5], line 2, in inverse_assert(x)
      1 def inverse_assert(x):
----> 2     assert type(x) == float or type(x) ==int
      3     return 1/x

AssertionError:
```

Notice this returns an `AssertionError`, but no informative message. Traps error correctly, but could be more informative to the user. Rewrite this to include a message as:

In [9]:
```
def inverse_assert(x):
    assert type(x) == float or type(x) ==int, 'input x should be a float or int'
    return 1/x
```

In [10]:
```
inverse_assert(5)
```

Out[10]:
```
0.2
```

In [11]:
```
inverse_assert(2.5)
```

Out[11]:
```
0.4
```

In [12]:
```
inverse_assert('frog')
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[12], line 1
----> 1 inverse_assert('frog')

Cell In[9], line 2, in inverse_assert(x)
      1 def inverse_assert(x):
----> 2     assert type(x) == float or type(x) ==int, 'input x should be a float or int'
      3     return 1/x

AssertionError: input x should be a float or int
```

Now this gives helpful feedback

In [13]:
```
inverse_assert(0)
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[13], line 1
----> 1 inverse_assert(0)

Cell In[9], line 3, in inverse_assert(x)
      1 def inverse_assert(x):
      2     assert type(x) == float or type(x) ==int, 'input x should be a float or int'
----> 3     return 1/x

ZeroDivisionError: division by zero
```

Also doesn't work. Add an assertion to trap zero input:

In [15]:
```
def inverse_assert(x):
    assert type(x) == float or type(x) ==int, 'input x should be a float or int'
    assert x!= 0, 'Input x should be nonzero'
    return 1/x
```

In [16]:
```
inverse_assert(5)
```

Out[16]:
```
0.2
```

In [17]:
```
inverse_assert(2.5)
```

Out[17]:
```
0.4
```

In [18]:
```
inverse_assert('frog')
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[18], line 1
----> 1 inverse_assert('frog')

Cell In[15], line 2, in inverse_assert(x)
      1 def inverse_assert(x):
----> 2     assert type(x) == float or type(x) ==int, 'input x should be a float or int'
      3     assert x!= 0, 'Input x should be nonzero'
      4     return 1/x

AssertionError: input x should be a float or int
```

In [19]:
```
inverse_assert(0)
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[19], line 1
----> 1 inverse_assert(0)

Cell In[15], line 3, in inverse_assert(x)
      1 def inverse_assert(x):
      2     assert type(x) == float or type(x) ==int, 'input x should be a float or int'
----> 3     assert x!= 0, 'Input x should be nonzero'
      4     return 1/x

AssertionError: Input x should be nonzero
```

This is the "defensive programming" style: we check the inputs to be valid for what we want to do, trap invalid input using assertions, and otherwise compute what we want

The second approach that we can use in Python is exceptions. Python allows us to "catch" errors and provide feedback based on them.

In [20]:
```
def inverse_except(x):
    try:
        quotient = 1/x
    except:
        quotient = None
    return quotient
```

In [21]:
```
inverse_except(5)
```

Out[21]:
```
0.2
```

In [22]:
```
inverse_except(2.5)
```

Out[22]:
```
0.4
```

In [23]:
```
inverse_except('frog')
```

In [24]:
```
inverse_except(0)
```

This is good, in that it runs, but not so good, in that it provides no feedback.

Notice the difference between

In [25]:
```
1/'frog'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[25], line 1
----> 1 1/'frog'

TypeError: unsupported operand type(s) for /: 'int' and 'str'
```

In [26]:
```
1/0
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[26], line 1
----> 1 1/0

ZeroDivisionError: division by zero
```

Can use specific errors to provide feedback

In [27]:
```
def inverse_except2(x):
    quotient = None
    try:
        quotient = 1/x
    except ZeroDivisionError:
        print("Cannot divide by zero")
    except TypeError:
        print("Cannot divide by type", type(x))
    return quotient
```

In [28]:
```
inverse_except2(5)
```

Out[28]:
```
0.2
```

In [29]:
```
inverse_except2(2.5)
```

Out[29]:
```
0.4
```

In [30]:
```
inverse_except2('frog')
```

```
Cannot divide by type <class 'str'>
```

In [31]:
```
inverse_except2(0)
```

```
Cannot divide by zero
```

Python has many built-in Exception types that we can use to direct this behaviour.

In general, `try-except` blocks are more useful in writing your code (check for errors and act accordingly), while `assert` are more useful in writing your tests.

When we test code, we want to test it in the smallest possible "units". Call this "unit testing".

What is a "unit" of code?

- An individual function or method is generally considered a single unit

Functions, of course, can be short and simple or long and complex.

Good code should have appropriately short functions

- They should do *something*
- Short functions are easier to understand, to test, to reuse.

Let's look at a unit of code, in `array_stats.py`

Simple code to compute the average of a list of numbers. A test for this code is just a function

In [7]:
```
import array_stats

def test_zero():
    data_list = [-2, -1, 0, 1, 2]
    observed = array_stats.mean(data_list)
    expected = 0
    assert observed == expected, "Computed mean did not match expected value"
```

In [8]:
```
test_zero()
```

Can write more tests:

In [13]:
```
def test_floatvalue():
    data_list = [1, 2, 3, 4]
    observed = array_stats.mean(data_list)
    expected = 2.5
    assert observed == expected, "Computed mean did not match expected value"
```

In [14]:
```
test_floatvalue()
```

In [15]:
```
def test_big():
    data_list = range(1, 1000)
    observed = array_stats.mean(data_list)
    expected = 500
    assert observed == expected, "Computed mean did not match expected value"
```

In [16]:
```
test_big()
```

Aside: `range` is a way of generating sequences in Python. However, it returns an object of type `range`, rather than an array or list.

What is `range(1,1000)` ?

In [17]:
```
print(range(1,5))
```

```
range(1, 5)
```

Not very informative. Can, however, use ranges to define loop indices:

In [18]:
```
for i in range(1,5):
    print(i)
```

```
1
2
3
4
```

Gives us sequence 1, 2, 3, 4, whose mean is 5/2. `range(1,1000)` would give us 1, 2, ..., 999, whose mean is 500

Range objects are useful, even if they're not so easy to relate to other types. We're going to see these again when we cover looping.

Idea is that `range(start,stop,step)` gives us an iterator (like a list, but different) with entries

```
start, start+step, start+2*step, ...
```

up to, but not including (or exceeding) `stop`.

If we omit one argument, get `range(start, stop)` with `step=1`. If we omit two arguments, get `range(stop)` with `start=0` and `step=1`.

Back to testing. We've now written 3 meaningful tests of our function to compute the mean: one that gives answer zero, one that gives a float value, and one that acts on a large dataset defined through `range`. Note that all these tests have integer or simple averages and integer data.

We could write more and more tests. Try one more, to see that our function works when the input data is float valued, instead of integers

In [20]:
```
def test_floatarray():
    data_list = [0.5, 1.2, 3.8, 4.5]
    observed = array_stats.mean(data_list)
    expected = 2.5
    assert observed == expected, "Computed mean did not match expected value"
```

In [21]:
```
test_floatarray()
```

We could continue to accumulate tests, but gets tedious to do this manually.

It would be better to have an automatic way to run tests. Our tool for this is `pytest`

Simplest way to use pytest is to just collect testing functions into a script. See `test_array_stats.py`

Run `pytest` at command line, either as just `pytest` or `python -m pytest` (in general, can run `python -m module_name`). Need to have installed `pytest` first, which may or may not be already done as part of your python library. If needed, run `pip install pytest` or `python -m pip install pytest` first.

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest test_array_stats.py
============================================ test session starts ============================================
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0
collected 4 items

test_array_stats.py ....                                                                             [100%]

============================================= 4 passed in 0.02s =============================================
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan%
```

See that this says that it ran 4 tests from `test_array_stats.py`. What happened?

In `test_array_stats.py`, there are 4 functions, each of these is identified as a "test". `pytest` reports that all tests passed, meaning each function ran successfully to completion.

Can use `-v` to get more details:

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest -v test_array_stats.py
============================================ test session starts ============================================
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0 -- /Users/smaclachlan/firedrake-June2025/bin/python3.13
cachedir: .pytest_cache
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0
collected 4 items

test_array_stats.py::test_zero PASSED                                                                [ 25%]
test_array_stats.py::test_float_value PASSED                                                         [ 50%]
test_array_stats.py::test_big PASSED                                                                 [ 75%]
test_array_stats.py::test_float_array PASSED                                                         [100%]

============================================= 4 passed in 0.02s =============================================
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan%
```

Change one of our tests so that it fails

See red "FAILED" notification, and details on failed test:

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest -v test_array_stats.py
============================================ test session starts ============================================
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0 -- /Users/smaclachlan/firedrake-June2025/bin/python3.13
cachedir: .pytest_cache
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0
collected 4 items

test_array_stats.py::test_zero FAILED                                                                [ 25%]
test_array_stats.py::test_float_value PASSED                                                         [ 50%]
test_array_stats.py::test_big PASSED                                                                 [ 75%]
test_array_stats.py::test_float_array PASSED                                                         [100%]

================================================= FAILURES ==================================================
_________________________________________________ test_zero _________________________________________________

    def test_zero():
        data_list = [-2, -1, 0, 1, 2]
        observed = array_stats.mean(data_list)
        expected = 1
>       assert observed == expected
E       assert 0.0 == 1

test_array_stats.py:8: AssertionError
========================================== short test summary info ==========================================
FAILED test_array_stats.py::test_zero - assert 0.0 == 1
======================================== 1 failed, 3 passed in 0.06s ========================================
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan%
```

Run without `-v` flag:

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest test_array_stats.py
============================================ test session starts ============================================
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0
collected 4 items

test_array_stats.py F...                                                                             [100%]

================================================= FAILURES ==================================================
_________________________________________________ test_zero _________________________________________________

    def test_zero():
        data_list = [-2, -1, 0, 1, 2]
        observed = array_stats.mean(data_list)
        expected = 1
>       assert observed == expected
E       assert 0.0 == 1

test_array_stats.py:8: AssertionError
========================================== short test summary info ==========================================
FAILED test_array_stats.py::test_zero - assert 0.0 == 1
======================================== 1 failed, 3 passed in 0.05s ========================================
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan%
```

- Failed tests get red "F", same output from them

When we write tests in this style, there's a lot of repetition. Can avoid this by "parametrizing" tests. Look at `test_array_stats_pytest.py`

Run this code and output is quite similar:

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest test_array_stats_pytest.py
============================================ test session starts ============================================
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0
collected 4 items

test_array_stats_pytest.py ....                                                                      [100%]

============================================= 4 passed in 0.02s =============================================
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest -v test_array_stats_pytest.py
============================================ test session starts ============================================
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0 -- /Users/smaclachlan/firedrake-June2025/bin/python3.13
cachedir: .pytest_cache
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0
collected 4 items

test_array_stats_pytest.py::test_mean[data_list0-0] PASSED                                           [ 25%]
test_array_stats_pytest.py::test_mean[data_list1-2.5] PASSED                                         [ 50%]
test_array_stats_pytest.py::test_mean[data_list2-500] PASSED                                         [ 75%]
test_array_stats_pytest.py::test_mean[data_list3-2.5] PASSED                                         [100%]

============================================= 4 passed in 0.02s =============================================
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan%
```

When a test case fails, get very similar output as when we don't parametrize the tests:

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest -v test_array_stats_pytest.py
============================================ test session starts ============================================
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0 -- /Users/smaclachlan/firedrake-June2025/bin/python3.13
cachedir: .pytest_cache
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0
collected 4 items

test_array_stats_pytest.py::test_mean[data_list0-0] PASSED                                           [ 25%]
test_array_stats_pytest.py::test_mean[data_list1-2.5] PASSED                                         [ 50%]
test_array_stats_pytest.py::test_mean[data_list2-500] PASSED                                         [ 75%]
test_array_stats_pytest.py::test_mean[data_list3-2.6] FAILED                                         [100%]

================================================= FAILURES ==================================================
_________________________________________ test_mean[data_list3-2.6] _________________________________________

data_list = [0.5, 1.2, 3.8, 4.5], expected = 2.6

    @pytest.mark.parametrize("data_list,expected",
                             [([-2, -1, 0, 1, 2], 0),
                              ([1, 2, 3, 4], 2.5),
                              (range(1, 1000), 500),
                              ([0.5, 1.2, 3.8, 4.5], 2.6)])
    def test_mean(data_list, expected):
        observed = array_stats.mean(data_list)
>       assert observed == expected
E       assert 2.5 == 2.6

test_array_stats_pytest.py:12: AssertionError
========================================== short test summary info ==========================================
FAILED test_array_stats_pytest.py::test_mean[data_list3-2.6] - assert 2.5 == 2.6
======================================== 1 failed, 3 passed in 0.05s ========================================
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan%
```

Can separately parametrize variables if we want to test all possible combinations. Look at `combo_test.py`

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest -v combo_test.py
============================================ test session starts ============================================
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0 -- /Users/smaclachlan/firedrake-June2025/bin/python3.13
cachedir: .pytest_cache
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0
collected 4 items

combo_test.py::test_sum[2-0] PASSED                                                                  [ 25%]
combo_test.py::test_sum[2-1] FAILED                                                                  [ 50%]
combo_test.py::test_sum[3-0] PASSED                                                                  [ 75%]
combo_test.py::test_sum[3-1] FAILED                                                                  [100%]

================================================= FAILURES ==================================================
_______________________________________________ test_sum[2-1] _______________________________________________

x = 1, y = 2

    @pytest.mark.parametrize("x", [0, 1])
    @pytest.mark.parametrize("y", [2, 3])
    def test_sum(x, y):
>       assert x+y == y
E       assert (1 + 2) == 2

combo_test.py:7: AssertionError
_______________________________________________ test_sum[3-1] _______________________________________________

x = 1, y = 3

    @pytest.mark.parametrize("x", [0, 1])
    @pytest.mark.parametrize("y", [2, 3])
    def test_sum(x, y):
>       assert x+y == y
E       assert (1 + 3) == 3

combo_test.py:7: AssertionError
========================================== short test summary info ==========================================
FAILED combo_test.py::test_sum[2-1] - assert (1 + 2) == 2
FAILED combo_test.py::test_sum[3-1] - assert (1 + 3) == 3
======================================== 2 failed, 2 passed in 0.05s ========================================
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan%
```

Output shows y and x values for each test (read `parametrize` statements from the bottom upwards) and that some tests have failed (as expected).

For expected fails (e.g., when you write the test before you write the code, or are waiting for someone else to fix a bug), you can mark a test with an "expected fail" or `xfail`, to tell pytest that this "doesn't count" as a real failure.

Replace one of the values with `pytest.param(1, marks=pytest.mark.xfail)` and re-run:

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest -v combo_test.py
============================================ test session starts ============================================
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0 -- /Users/smaclachlan/firedrake-June2025/bin/python3.13
cachedir: .pytest_cache
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0
collected 4 items

combo_test.py::test_sum[2-0] PASSED                                                                  [ 25%]
combo_test.py::test_sum[2-1] XFAIL                                                                   [ 50%]
combo_test.py::test_sum[3-0] PASSED                                                                  [ 75%]
combo_test.py::test_sum[3-1] XFAIL                                                                   [100%]

======================================== 2 passed, 2 xfailed in 0.03s ========================================
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan%
```

Now see "XFAIL" instead of "FAILURE" and no "failed" output. If we run without `-v`, just see `x` instead of `F`:

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest combo_test.py
============================================ test session starts ============================================
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0
collected 4 items

combo_test.py .x.x                                                                                   [100%]

======================================== 2 passed, 2 xfailed in 0.03s ========================================
```

We can also specify inputs to tests using "fixtures", functions that return inputs when evaluated:

```
[math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% pytest -v combo_test.py
============================================ test session starts ============================================
platform darwin -- Python 3.13.3, pytest-8.3.5, pluggy-1.6.0 -- /Users/smaclachlan/firedrake-June2025/bin/python3.13
cachedir: .pytest_cache
rootdir: /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes
plugins: mpi-pytest-2025.5.0
collected 6 items

combo_test.py::test_sum[2-0] PASSED                                                                  [ 16%]
combo_test.py::test_sum[2-1] XFAIL                                                                   [ 33%]
combo_test.py::test_sum[3-0] PASSED                                                                  [ 50%]
combo_test.py::test_sum[3-1] XFAIL                                                                   [ 66%]
combo_test.py::test_sum2[2] PASSED                                                                   [ 83%]
combo_test.py::test_sum2[3] PASSED                                                                   [100%]

======================================== 4 passed, 2 xfailed in 0.03s ========================================
```

Fixtures are useful when your input to a test needs to be more complicated than a simple piece of data (number, short list). For example, when you want to have an item with a complex data type or object